

DAY 6

1. Introduction to Shell Parameters

A **shell parameter** is an entity used by the shell to **store or represent a value (data)** that can be referenced later.

In shell scripting, parameters allow us to work with dynamic data instead of hardcoding values everywhere.

Types of Shell Parameters

Shell parameters are broadly divided into **three categories**:

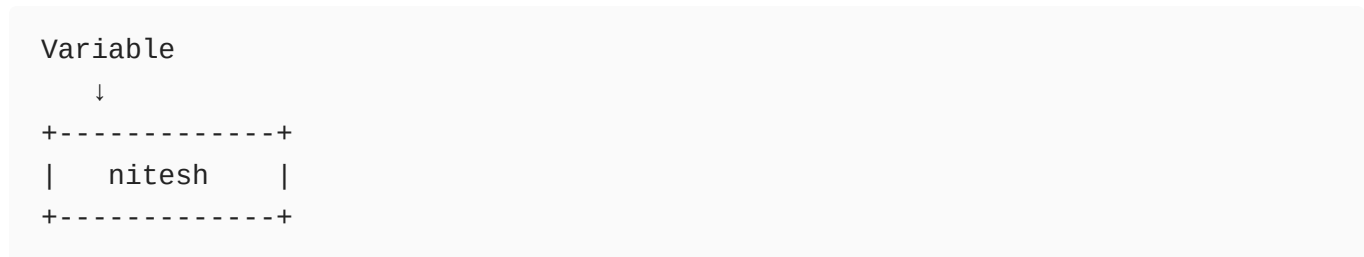
1. **Variables**
2. **Positional Parameters**
3. **Special Parameters**

For now, the main focus is on **variables**.

2. What is a Variable?

A **variable** is like a **container/box** that **stores a value**.

Think of it like this:



For example:

```
student=nitesh
```

Here:

- `student` → variable name
- `nitesh` → value stored in the variable

We can later access the stored value using the variable.

```
echo "${student}"
```

Output:

```
nitesh
```

Basic Concept

```
Assign value
  ↓
student=nitesh
  ↓
Store "nitesh" in student
  ↓
Reference variable
  ↓
${student}
  ↓
nitesh
```

3. Why Do We Use Variables?

Variables make shell scripts:

- **Dynamic**
- **Reusable**
- **Easier to maintain**
- **Easier to modify**
- **Less repetitive**

Without a variable

Suppose you repeatedly need to use a username:

```
echo "Hello nitesh"
echo "nitesh is learning Linux"
echo "nitesh is running a script"
```

If the username changes, you need to modify every line.

With a variable

```
student=nitesh

echo "Hello ${student}"
echo "${student} is learning Linux"
echo "${student} is running a script"
```

Now you only need to change:

```
student=sourya
```

The rest of the script automatically uses the new value.

4. Variable Assignment

The basic syntax for creating a user-defined variable is:

```
variable_name=value
```

Example:

```
student=nitesh
```

Another example:

```
age=21
```

Another:

```
course=Linux
```

Important Rule: No Spaces Around =

Correct:

```
student=nitesh
```

Incorrect:

```
student = nitesh
```

Incorrect:

```
student= nitesh
```

Incorrect:

```
student =nitesh
```

The shell interprets spaces as command/argument separators.

Therefore:

```
student=nitesh
```

is an **assignment**, whereas:

```
student = nitesh
```

is interpreted differently by the shell and will generally result in an error.

5. Referencing a Variable

After assigning a value, we need a way to retrieve it.

We use `$` before the variable name.

For example:

```
student=nitesh  
echo $student
```

Output:

```
nitesh
```

A more professional and safer form is:

```
echo "${student}"
```

Recommended Syntax

```
${variable_name}
```

Example:

```
name=Sourya  
echo "${name}"
```

Output:

```
Sourya
```

6. Why Use `${variable_name}` ?

You will often see both:

```
$student
```

and

```
${student}
```

Both can reference a variable.

However, `${...}` clearly tells the shell where the variable name ends.

For example:

```
name=Sourya
echo "${name}Dutta"
```

Output:

```
SouryaDutta
```

The braces make it clear that the variable is:

```
name
```

rather than:

```
nameDutta
```

Therefore, a good shell-scripting habit is:

```
"${variable_name}"
```

especially when combining variables with other text.

7. User-Defined Variables

A **user-defined variable** is a variable that you create yourself inside your shell session or script.

Example:

```
name=Sourya
age=21
course=Cybersecurity
```

These variables are created by the user according to the requirements of the script.

Example

```
name=Sourya
course=Cybersecurity

echo "Name: ${name}"
echo "Course: ${course}"
```

Output:

```
Name: Sourya
Course: Cybersecurity
```

8. Rules for Naming Variables

Shell variables have certain naming rules.

Rule 1 – The name should begin with a letter or underscore

Valid:

```
student=nitesh
_name=value
student_name=nitesh
```

Invalid:

```
1student=nitesh
```

A variable name should **not start with a number**.

Rule 2 – Numbers can be used after the first character

Valid:

```
student1=nitesh
student2026=nitesh
```

But:

```
1student=nitesh
```

is invalid.

Rule 3 – Avoid special characters in variable names

For example:

```
student-name=nitesh
```

is not a valid normal variable name because `-` has a special meaning in shell syntax.

Prefer:

```
student_name=nitesh
```

Rule 4 – No spaces in the variable assignment

Correct:

```
student_name=nitesh
```

Incorrect:

```
student_name = nitesh
```

9. Variables Are Changeable

One important characteristic of variables is that their values can be changed.

For example:


```
student=nitesh  
echo "${student}"
```

Output:

```
nitesh
```

Now change it:

```
student=sourya  
echo "${student}"
```

Output:

```
sourya
```

The same variable now contains a different value.

```
student  
  ↓  
nitesh  
  ↓  
change value  
  ↓  
sourya
```

This is why variables are useful for storing information that may change during execution.

10. System-Defined / Environment Variables

Not all variables are created manually by the user.

The operating system and shell also provide predefined variables.

These are commonly called **environment variables** when they are part of the process environment.

They contain information about the current system, user, shell, directories, configuration, etc.

Some commonly encountered variables are:

Variable	Purpose
\$HOME	User's home directory
\$USER	Current username
\$HOSTNAME	Host/computer name
\$SHELL	User's default shell
\$PWD	Current working directory

11. \$HOME

\$HOME contains the path to the current user's **home directory**.

Check it using:

```
echo "${HOME}"
```

Example output:

```
/home/sourya
```

The exact path depends on the user.

Why is \$HOME useful?

Instead of hardcoding:

```
cd /home/sourya
```

you can use:

```
cd "${HOME}"
```

This makes scripts more portable between users.

12. \$USER

`$USER` generally contains the username of the current user.

```
echo "${USER}"
```

Example:

```
sourya
```

This can be useful in scripts where you need to know which user is executing the command.

Example:

```
echo "Current user: ${USER}"
```

13. \$HOSTNAME

`$HOSTNAME` contains the hostname of the machine.

```
echo "${HOSTNAME}"
```

Example:

```
cyber30
```

The value depends on the computer's configured hostname.

This is particularly useful when working with:

- Multiple Linux machines
- Servers
- Virtual machines
- Containers
- Network environments

Example:

```
echo "This script is running on ${HOSTNAME}"
```

14. \$SHELL

`$SHELL` indicates the user's configured/default shell.

Example:

```
echo "${SHELL}"
```

Possible output:

```
/bin/bash
```

or:

```
/bin/zsh
```

depending on the user's configuration.

Important distinction

`$SHELL` generally tells you the user's **configured login shell**. It does not necessarily prove which shell is currently interpreting every command.

For example, you might have:

```
$SHELL  
/bin/bash
```

but explicitly run a script with another shell.

15. \$PWD

`$PWD` represents the **Present Working Directory**.

You can check it using:

```
echo "${PWD}"
```

Example:

```
/home/sourya/Documents
```

It generally corresponds to the directory shown by:

```
pwd
```

For example:

```
$ pwd
/home/sourya/Documents

$ echo "${PWD}"
/home/sourya/Documents
```

16. Checking Environment Variables

You can inspect environment variables using commands such as:

```
printenv
```

or:

```
env
```

For a specific variable:

```
printenv HOME
```

or:

```
echo "${HOME}"
```

You can also use:

```
echo "${USER}"
echo "${HOSTNAME}"
echo "${SHELL}"
echo "${PWD}"
```

17. User-Defined vs Environment Variables

A useful distinction:

Feature	User-Defined Variable	Environment Variable
Created by	User/script	Shell/system/environment
Example	name=Sourya	HOME=/home/sourya
Purpose	Store data needed by script	Provide environment information/configuration
Automatically available	Not necessarily	Available to processes when exported
Can change?	Yes	Many can be changed, subject to context

Important concept: `export`

A shell variable does not automatically become an environment variable.

For example:

```
name=Sourya
```

creates a shell variable.

To make it available to child processes:

```
export name
```

or:

```
export name=Sourya
```

Then child processes can inherit it.

This distinction becomes important when learning **processes, environment variables, and shell scripting**.

18. Shell Variable vs Environment Variable

Think of it like this:

```
Shell
|
├── Shell Variables
│   └── name=Sourya
└── Environment Variables
    ├── HOME=/home/sourya
    ├── USER=sourya
    ├── SHELL=/bin/bash
    └── PWD=/home/sourya
```

When a variable is exported:

```
export name=Sourya
```

it becomes part of the environment inherited by child processes.

19. Example: Putting Everything Together

```
#!/bin/bash

student="Sourya"
course="Cybersecurity"

echo "Student: ${student}"
echo "Course: ${course}"

echo "Username: ${USER}"
```

```
echo "Home Directory: ${HOME}"  
echo "Hostname: ${HOSTNAME}"  
echo "Shell: ${SHELL}"  
echo "Current Directory: ${PWD}"
```

Possible output:

```
Student: Sourya  
Course: Cybersecurity  
Username: sourya  
Home Directory: /home/sourya  
Hostname: cyber30  
Shell: /bin/bash  
Current Directory: /home/sourya
```

Notice that:

```
student  
course
```

are **user-defined variables**, while:

```
USER  
HOME  
HOSTNAME  
SHELL  
PWD
```

are commonly provided by the shell/environment.
